sive, we wouldn't need comments at all. But that is not the world that we live in. Plain code just isn't expressive enough to explain itself. Sometime s, code handle very complex logics that forces us to use comments. Every time you find the need to write a comment, stop and check if there is a better way to express yourself through the code.

The primary reason to avoid comments is because overtime they become very misleading. Code changes, it is refactored and evolved. However, comments aren't. If it is tough to maintain code, it is a lot tougher to maintain comments. Programmers can't realistically maintain them. Often, they get separated from the code they were meant to explain.

Very often, we write code that we know is messy and disorganised. Rather than write comments to explain the code, the best practice is to structure and improve the code.

Good comments

Some comments are beneficial and necessary. For example,

- **Legal comments:**

Comments that we are forced to write for legal reasons so as to follow our corporate coding standards.

eg.

```
// Copyright (C) 2003,2004,2005 by Object Mentor, Inc. All rights reserved.
// Released under the terms of the GNU General Public License version 2 or later.
```

- **Informative comments**

It is sometimes a good practice to write comments that convey basic information.

eg.

```
// format matched kk:mm:ss EEE, MMM dd, yyyy
Pattern timeMatcher =
Pattern.compile("\\d*:\\d*:\\d* \\w*, \\w* \\d*, \\d*");
```

In this case the comment lets us know that the regular expression is intended to match a time and date that were formatted with the SimpleDateFormat.format function using the specified format string.

- **Clarification**

Comments may be used to explain a complex expression or express the return value of a statement in a simpler way.